Experiment No. 1

Implementation of Fundamental Image Processing Operations
using Python and OpenCV

Aim

To develop a comprehensive understanding of fundamental image processing techniques by implementing image acquisition, storage, color space conversion, geometric transformations, and image enhancement operations using Python and OpenCV, thereby establishing the foundation for advanced computer vision applications.

Purpose

Image processing forms the core of modern computer vision systems used in autonomous driving, healthcare, surveillance, robotics, and industrial automation. Before advanced tasks such as feature extraction, object detection, or image classification, it is essential to understand how digital images are represented, manipulated, and transformed. This experiment implements fundamental operations (read, write, convert, resize, and manipulate images) and studies how they influence image quality and prepare images for subsequent computer vision tasks.

Prerequisites

- Basic knowledge of Python programming
- Familiarity with digital image representation (pixels, resolution, color channels)
- Python libraries: OpenCV (cv2), NumPy, and Matplotlib
- Jupyter Notebook, Google Colab, or Visual Studio Code

Steps (Procedure)

1. Install and import the required libraries (OpenCV, NumPy, Matplotlib).
2. Load a color image from the local system and display it using both OpenCV and Matplotlib.
3. Examine image properties: dimensions, resolution, number of channels, and data type.
4. Save the image in different formats (JPEG and PNG) and compare the outputs.
5. Convert the image from BGR/RGB to Grayscale, HSV, and LAB color spaces.
6. Perform geometric transformations: resizing, rotation, horizontal/vertical flipping.
7. Generate the complement (negative) by inverting pixel intensity values.
8. Crop a Region of Interest (ROI) and analyze its characteristics.
9. Display the original and processed images for comparison and observe the effects.
10. Document the observations and discuss the significance of these preprocessing techniques.

Observations

- JPEG is lossy and usually much smaller than PNG, which is lossless.
- Grayscale has 1 channel instead of 3, reducing data to one third.
- OpenCV stores channels in BGR order; images must be converted to RGB before display with Matplotlib.

- HSV separates color (Hue, Saturation) from brightness (Value); LAB separates lightness (L) from color (a, b).
- Rotation by a non-90 degree angle clips the corners of the image unless the canvas is enlarged.
- Flipping changes orientation only; pixel values are unchanged.
- The negative is computed as 255 - pixel; dark regions become bright and vice versa.
- Cropping an ROI discards pixels but keeps the original values unchanged (no interpolation).

Source Code

File: experiment1_image_processing.py

```
"""
Experiment 1: Fundamental Image Processing Operations using Python and OpenCV
Install:  pip install opencv-python numpy matplotlib
Usage:    python experiment1_image_processing.py [path_to_image]
"""
import os
import sys
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------------------------------
# Step 2: Load a colour image (OpenCV loads as BGR)
# ------------------------------------------------------------
path = sys.argv[1] if len(sys.argv) > 1 else "input.jpg"
img_bgr = cv2.imread(path)

if img_bgr is None:  # fallback so the script always runs
    print(f"'{path}' not found - generating a synthetic test image instead.")
    img_bgr = np.zeros((300, 400, 3), dtype=np.uint8)
    img_bgr[:, :133] = (255, 0, 0)       # blue band (BGR)
    img_bgr[:, 133:266] = (0, 255, 0)    # green band
    img_bgr[:, 266:] = (0, 0, 255)       # red band
    cv2.circle(img_bgr, (200, 150), 70, (255, 255, 255), -1)
    cv2.putText(img_bgr, "OpenCV", (130, 160), cv2.FONT_HERSHEY_SIMPLEX,
                1.2, (0, 0, 0), 3)

img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# Display with Matplotlib (needs RGB) - OpenCV window shown at the end
plt.figure(figsize=(5, 4))
plt.imshow(img_rgb)
plt.title("Original (displayed via Matplotlib)")
plt.axis("off")
plt.show()

# cv2.imshow("Original (OpenCV)", img_bgr); cv2.waitKey(0); cv2.destroyAllWindows()

# ------------------------------------------------------------
# Step 3: Image properties
# ------------------------------------------------------------
h, w, c = img_bgr.shape
print("---- Image Properties ----")
print(f"Height x Width : {h} x {w}")
print(f"Channels       : {c}")
print(f"Total pixels   : {img_bgr.size // c}")
print(f"Data type      : {img_bgr.dtype}")
print(f"Min/Max value  : {img_bgr.min()} / {img_bgr.max()}")

# ------------------------------------------------------------
# Step 4: Save in different formats and compare
# ------------------------------------------------------------
os.makedirs("output", exist_ok=True)
cv2.imwrite("output/saved.jpg", img_bgr, [cv2.IMWRITE_JPEG_QUALITY, 90])  # lossy
cv2.imwrite("output/saved.png", img_bgr, [cv2.IMWRITE_PNG_COMPRESSION, 3])  # lossless
print("\n---- File size comparison ----")
for f in ("saved.jpg", "saved.png"):
    print(f"{f}: {os.path.getsize('output/' + f) / 1024:.1f} KB")

# ------------------------------------------------------------
# Step 5: Colour space conversions
# ------------------------------------------------------------
gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)
print("\nGray shape:", gray.shape, "| HSV shape:", hsv.shape, "| LAB shape:", lab.shape)

# ------------------------------------------------------------
# Step 6: Geometric transformations
# ------------------------------------------------------------
resized = cv2.resize(img_bgr, (w // 2, h // 2), interpolation=cv2.INTER_AREA)
scaled_up = cv2.resize(img_bgr, None, fx=1.5, fy=1.5, interpolation=cv2.INTER_LINEAR)

M = cv2.getRotationMatrix2D((w / 2, h / 2), 45, 1.0)  # centre, angle, scale
rotated = cv2.warpAffine(img_bgr, M, (w, h))

flip_h = cv2.flip(img_bgr, 1)   # horizontal
flip_v = cv2.flip(img_bgr, 0)   # vertical
flip_b = cv2.flip(img_bgr, -1)  # both
```

```
print(f"Resized (half): {resized.shape} | Scaled (1.5x): {scaled_up.shape}")

# ---------------------------------------------------------------
# Step 7: Complement (negative)
# ---------------------------------------------------------------
negative = cv2.bitwise_not(img_bgr)          # same as 255 - img
negative_gray = 255 - gray

# ---------------------------------------------------------------
# Step 8: Crop ROI (NumPy slicing: [y1:y2, x1:x2])
# ---------------------------------------------------------------
y1, y2 = h // 4, 3 * h // 4
x1, x2 = w // 4, 3 * w // 4
roi = img_bgr[y1:y2, x1:x2]
print("\n---- ROI characteristics ----")
print(f"ROI shape : {roi.shape}")
print(f"Mean BGR  : {roi.reshape(-1, 3).mean(axis=0).round(2)}")
print(f"Std  BGR  : {roi.reshape(-1, 3).std(axis=0).round(2)}")

# ---------------------------------------------------------------
# Step 9: Display original vs processed
# ---------------------------------------------------------------
def rgb(x):
    return cv2.cvtColor(x, cv2.COLOR_BGR2RGB)

panels = [
    ("Original", rgb(img_bgr), None),
    ("Grayscale", gray, "gray"),
    ("HSV (as RGB view)", rgb(hsv), None),
    ("LAB (as RGB view)", rgb(lab), None),
    ("Resized (0.5x)", rgb(resized), None),
    ("Rotated 45°", rgb(rotated), None),
    ("Flip Horizontal", rgb(flip_h), None),
    ("Flip Vertical", rgb(flip_v), None),
    ("Negative", rgb(negative), None),
    ("ROI Crop", rgb(roi), None),
    ("Gray Negative", negative_gray, "gray"),
    ("Flip Both", rgb(flip_b), None),
]

plt.figure(figsize=(14, 10))
for i, (title, im, cmap) in enumerate(panels, 1):
    plt.subplot(3, 4, i)
    plt.imshow(im, cmap=cmap)
    plt.title(title, fontsize=9)
    plt.axis("off")
plt.tight_layout()
plt.savefig("output/comparison.png", dpi=150)
plt.show()

# Bonus: show individual HSV / LAB channels
fig, ax = plt.subplots(2, 3, figsize=(10, 6))
for a, ch, t in zip(ax[0], cv2.split(hsv), ["H", "S", "V"]):
    a.imshow(ch, cmap="gray"); a.set_title(f"HSV - {t}"); a.axis("off")
for a, ch, t in zip(ax[1], cv2.split(lab), ["L", "a", "b"]):
    a.imshow(ch, cmap="gray"); a.set_title(f"LAB - {t}"); a.axis("off")
plt.tight_layout()
plt.show()
```

Questions and Answers

1. What is a digital image? Differentiate between grayscale and color images.
A digital image is a 2D grid of pixels, each holding a numeric intensity, obtained by sampling and quantizing a continuous scene.

A **grayscale** image has one channel (0-255 for 8-bit) and shows only brightness. A **color** image has multiple channels (typically 3: R, G, B), so each pixel is a vector such as (R, G, B) and the array shape is H x W x 3.

2. Explain the difference between RGB, BGR, HSV, and LAB color spaces.

- **RGB:** additive model with Red, Green, Blue channels, used by displays and Matplotlib.
- **BGR:** the same channels in reverse order. OpenCV uses it by default for historical reasons, so convert before using Matplotlib.
- **HSV:** Hue (color type, 0-179 in OpenCV), Saturation (purity), Value (brightness). It separates color from brightness, which suits color-based segmentation.
- **LAB:** L is lightness, a is green-red, b is blue-yellow. It is device-independent and perceptually uniform, so Euclidean distance roughly matches perceived color difference.

3. What is the purpose of converting an image to grayscale before further processing?
It reduces data to one third (1 channel instead of 3), cutting computation and memory, and removes color information that many algorithms do not need. Edge detection, thresholding, feature detectors (SIFT, ORB), Hough transforms, and Haar face detection work on intensity changes. It also makes algorithms less sensitive to lighting color variation.

4. Explain the concept of image complement (negative image) and mention its practical applications.
Each pixel is replaced by L - 1 - r; for 8-bit images this is 255 - r. Dark areas become bright and vice versa. Applications:

- Enhancing white or gray detail in dark regions of medical images (X-rays, mammograms)
- Viewing photographic film negatives
- Highlighting features in satellite or document images
- Data augmentation

5. Differentiate between image resizing, cropping, and scaling.

- **Resizing:** changes the image to an exact target size (width x height), possibly altering the aspect ratio.
- **Scaling:** multiplies dimensions by a factor (fx, fy) such as 0.5x or 2x, usually preserving aspect ratio. In practice it is a type of resizing.
- **Cropping:** extracts a sub-region, so the field of view shrinks and pixels are discarded, but no interpolation happens and pixel values stay unchanged.

6. What is a Region of Interest (ROI), and why is it important in computer vision?
A Region of Interest is the sub-area of an image that contains the information to be analyzed (a face, a license plate, a tumor). Processing only the ROI reduces computation, removes background noise and distractions, and improves accuracy and speed. Examples include object detection, tracking, and OCR.

7. Why is OpenCV preferred over conventional image processing libraries for computer vision applications?

- Open source and free, with 2,500+ optimized algorithms written in C/C++ (with GPU/SIMD acceleration) for real-time performance
- Supports Python, C++, and Java, and works across platforms including mobile and embedded systems
- Integrates easily with NumPy and deep learning frameworks, and has a large community

- Covers a complete vision pipeline: filtering, features, calibration, tracking, and DNN inference

8. Explain how image resolution and pixel intensity influence image quality.

- **Resolution** (number of pixels) determines spatial detail. Higher resolution gives finer detail and sharper edges; low resolution gives blocky, pixelated images.
- **Pixel intensity (bit depth)** determines tonal detail. More gray levels (256 for 8-bit) give smooth gradients; fewer levels cause false contouring. The intensity distribution also sets contrast and brightness: a narrow histogram looks washed out, a wide one has good contrast.

9. Mention five real-world applications where basic image preprocessing is an essential step.

1. Medical imaging (noise removal and contrast enhancement in X-ray, MRI, CT)
2. Autonomous vehicles (lane and traffic-sign detection)
3. Face recognition and biometrics (alignment, grayscale, normalization)
4. Optical character recognition (binarization, deskewing)
5. Industrial inspection and surveillance (defect detection, license plate recognition)

10. How do image preprocessing techniques improve the performance of feature extraction and deep learning models?

- Noise removal and contrast enhancement make edges, corners, and textures clearer and more repeatable, so feature extraction (SIFT, HOG, ORB) is more reliable.
- Resizing and normalization give fixed-size, consistently scaled inputs that CNNs require, and speed up training convergence.
- Color-space conversion improves robustness to lighting changes.
- ROI cropping removes irrelevant background so models learn the relevant features.
- Augmentation (flip, rotate, crop) increases data variety and reduces overfitting.
- Overall effect: higher accuracy, faster training and inference, and better generalization.

Conclusion

Fundamental image processing operations (loading, saving, color conversion, geometric transformation, negation, and ROI extraction) were implemented using Python and OpenCV. These operations standardize and enhance images, forming the essential preprocessing stage for feature extraction, object detection, and deep learning pipelines.